

Exam Prep AI - Focused Notes

Generated on 09 June 2026

DYNAMIC PROGRAMMING

1. Definition

Dynamic programming is a method for solving complex problems by breaking them down into smaller subproblems, solving each subproblem only once, and storing the solutions to subproblems to avoid redundant computation.

2. Key Exam Points

- DIVIDE THE PROBLEM INTO SMALLER SUBPROBLEMS.

- Solve each subproblem only once and store the solutions.
- Use memoization to store the solutions to subproblems.
- Example: Fibonacci sequence, knapsack problem.

3. Important Steps / Algorithm

1. Identify the subproblems.
2. Solve each subproblem recursively.
3. Store the solutions to subproblems.
4. Use memoization to avoid redundant computation.

4. Frequently Asked Questions

1. What is dynamic programming?
2. How does dynamic programming work?
3. What are the benefits of dynamic programming?

5. Quick Revision Summary

- Break down complex problems into smaller subproblems.
- Solve each subproblem only once and store the solutions.

- USE MEMOIZATION TO AVOID REDUNDANT COMPUTATION.

BACKTRACKING

1. Definition

Backtracking is a problem-solving strategy that involves trying different options and undoing them if they lead to a dead end.

2. Key Exam Points

- Try different options and undo them if they lead to a dead end.

- USE RECURSION TO IMPLEMENT BACKTRACKING.

- Example: N-queens problem, Sudoku.

3. Important Steps / Algorithm

1. Try a solution.
2. If the solution is not valid, undo the last operation.

3. Try a different solution.
 4. Frequently Asked Questions
 1. What is backtracking?
 2. How does backtracking work?
 3. What are the benefits of backtracking?
 5. Quick Revision Summary
 - Try different options and undo them if they lead to a dead end.
-

- USE RECURSION TO IMPLEMENT BACKTRACKING.

GREEDY ALGORITHM

1. Definition

A greedy algorithm is a method for solving optimization problems by making the locally optimal choice at each step.
 2. Key Exam Points
-

- MAKE THE LOCALLY OPTIMAL CHOICE AT EACH STEP.

- Use a greedy approach to solve optimization problems.
 - Example: Huffman coding, activity selection problem.
3. Important Steps / Algorithm
 1. Identify the optimization problem.
 2. Choose the locally optimal solution at each step.
 3. Use a greedy approach to solve the problem.
 4. Frequently Asked Questions
 1. What is a greedy algorithm?
 2. How does a greedy algorithm work?
 3. What are the benefits of a greedy algorithm?
 5. Quick Revision Summary
-

- MAKE THE LOCALLY OPTIMAL CHOICE AT EACH STEP.

- Use a greedy approach to solve optimization problems.
-

DIVIDE AND CONQUER

1. Definition

Divide and conquer is a problem-solving strategy that involves dividing the problem into smaller subproblems, solving each subproblem, and combining the solutions to solve the original problem.
 2. Key Exam Points
-

- DIVIDE THE PROBLEM INTO SMALLER SUBPROBLEMS.

- SOLVE EACH SUBPROBLEM RECURSIVELY.

- Combine the solutions to solve the original problem.

- Example: Merge sort, binary search.

3. Important Steps / Algorithm

1. Divide the problem into smaller subproblems.
2. Solve each subproblem recursively.
3. Combine the solutions to solve the original problem.

4. Frequently Asked Questions

1. What is divide and conquer?
2. How does divide and conquer work?
3. What are the benefits of divide and conquer?

5. Quick Revision Summary

- DIVIDE THE PROBLEM INTO SMALLER SUBPROBLEMS.

- SOLVE EACH SUBPROBLEM RECURSIVELY.

- Combine the solutions to solve the original problem.

FIBONACCI SEQUENCE

1. Definition

The Fibonacci sequence is a series of numbers in which each number is the sum of the two preceding numbers.

2. Key Exam Points

- Each number is the sum of the two preceding numbers.
- Use dynamic programming to solve the Fibonacci sequence.
- Example: Fibonacci number generation.

3. Important Steps / Algorithm

1. Initialize the first two numbers.
2. Generate the next numbers recursively.
3. Use dynamic programming to avoid redundant computation.

4. Frequently Asked Questions

1. What is the Fibonacci sequence?
2. How does the Fibonacci sequence work?
3. What are the benefits of the Fibonacci sequence?

5. Quick Revision Summary

- Each number is the sum of the two preceding numbers.
- Use dynamic programming to solve the Fibonacci sequence.

PALINDROME

1. Definition

A palindrome is a string or number that reads the same backwards as forwards.

2. Key Exam Points

- Check if the string or number reads the same backwards as forwards.
- Use a recursive approach to solve the palindrome problem.
- Example: Palindrome number generation.

3. Important Steps / Algorithm

1. Check if the string or number is empty.
2. Compare the first and last characters.

3. If they are the same, return true.

4. Frequently Asked Questions

1. What is a palindrome?

2. How does a palindrome work?

3. What are the benefits of a palindrome?

5. Quick Revision Summary

- Check if the string or number reads the same backwards as forwards.
- Use a recursive approach to solve the palindrome problem.

RANDOMIZED APPROACH

1. Definition

A randomized approach is a problem-solving strategy that involves using random numbers or permutations to solve the problem.

2. Key Exam Points

- Use random numbers or permutations to solve the problem.
- Use a randomized algorithm to solve the problem.
- Example: Randomized number generation.

3. Important Steps / Algorithm

1. Generate random numbers or permutations.

2. Use the random numbers or permutations to solve the problem.

4. Frequently Asked Questions

1. What is a randomized approach?

2. How does a randomized approach work?

3. What are the benefits of a randomized approach?

5. Quick Revision Summary

- Use random numbers or permutations to solve the problem.
- Use a randomized algorithm to solve the problem.

HEURISTICS

1. Definition

Heuristics are rules of thumb that are used to make decisions or solve problems.

2. Key Exam Points

- Use heuristics to make decisions or solve problems.

- HEURISTICS ARE RULES OF THUMB.

- Example: Heuristic search algorithms.

3. Important Steps / Algorithm

1. Identify the problem.

2. Use a heuristic approach to solve the problem.

4. Frequently Asked Questions

1. What is a heuristic?

2. How does a heuristic work?

3. What are the benefits of a heuristic?

5. Quick Revision Summary

- Use heuristics to make decisions or solve problems.
-

- HEURISTICS ARE RULES OF THUMB.

MEANS-ENDS ANALYSIS

1. Definition

Means-ends analysis is a problem-solving strategy that involves breaking down the problem into smaller subproblems and solving each subproblem.

2. Key Exam Points

- Break down the problem into smaller subproblems.
-

- SOLVE EACH SUBPROBLEM RECURSIVELY.

- Example: Means-ends analysis algorithms.

3. Important Steps / Algorithm

1. Identify the subproblems.
2. Solve each subproblem recursively.
3. Combine the solutions to solve the original problem.

4. Frequently Asked Questions

1. What is means-ends analysis?
2. How does means-ends analysis work?
3. What are the benefits of means-ends analysis?

5. Quick Revision Summary

- Break down the problem into smaller subproblems.
-

- SOLVE EACH SUBPROBLEM RECURSIVELY.

PROBLEM-SOLVING STEPS

1. Definition

Problem-solving steps are the steps involved in solving a problem.

2. Key Exam Points

- IDENTIFY THE PROBLEM.

- UNDERSTAND THE PROBLEM.

- DEVELOP A SOLUTION.

- TEST THE SOLUTION.

- EVALUATE THE SOLUTION.

3. Important Steps / Algorithm

1. Identify the problem.
2. Understand the problem.
3. Develop a solution.
4. Test the solution.
5. Evaluate the solution.

4. Frequently Asked Questions

1. What are problem-solving steps?
 2. How do problem-solving steps work?
 3. What are the benefits of problem-solving steps?
5. Quick Revision Summary

- IDENTIFY THE PROBLEM.

- UNDERSTAND THE PROBLEM.

- DEVELOP A SOLUTION.

- TEST THE SOLUTION.

- EVALUATE THE SOLUTION.

BRUTE FORCE TECHNIQUE

1. Definition

The brute force technique is a problem-solving strategy that involves trying all possible solutions to a problem.

2. Key Exam Points

- TRY ALL POSSIBLE SOLUTIONS TO A PROBLEM.

- Use a brute force approach to solve the problem.
- Example: Brute force search algorithms.

3. Important Steps / Algorithm

1. Identify the possible solutions.
 2. Try each solution.
 3. Return the first solution that satisfies the problem.
- #### 4. Frequently Asked Questions
1. What is the brute force technique?
 2. How does the brute force technique work?
 3. What are the benefits of the brute force technique?
5. Quick Revision Summary

- TRY ALL POSSIBLE SOLUTIONS TO A PROBLEM.

- Use a brute force approach to solve the problem.
-

RECURSION

1. Definition

Recursion is a problem-solving strategy that involves breaking down the problem into smaller subproblems and solving each subproblem recursively.

2. Key Exam Points

- Break down the problem into smaller subproblems.
-

- SOLVE EACH SUBPROBLEM RECURSIVELY.

- Example: Recursive function calls.

3. Important Steps / Algorithm

1. Identify the subproblems.
2. Solve each subproblem recursively.
3. Combine the solutions to solve the original problem.

4. Frequently Asked Questions

1. What is recursion?
2. How does recursion work?
3. What are the benefits of recursion?

5. Quick Revision Summary

- Break down the problem into smaller subproblems.
-

- SOLVE EACH SUBPROBLEM RECURSIVELY.

DYNAMIC PROGRAMMING STRATEGIES

1. Definition

Dynamic programming strategies are methods for solving complex problems by breaking them down into smaller subproblems and solving each subproblem only once.

2. Key Exam Points

- Break down the problem into smaller subproblems.
 - Solve each subproblem only once and store the solutions.
-

- USE MEMOIZATION TO STORE THE SOLUTIONS.

- Example: Dynamic programming algorithms.

3. Important Steps / Algorithm

1. Identify the subproblems